Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе № 1

«Множества на основе битовых полей»

Выполнил:
студент группы 3825Б1ФИип2
Зотин И.А.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 5
3. Руководство программиста 6
3.1. Описание структуры программы 6
3.2. Описание структур данных 6
3.3. Описание алгоритмов 6
4. Результаты экспериментов и тестирования 10
Заключение 11
Литература 12
Приложение 13

Введение

В рамках данной лабораторной работы рассматривается реализация двух взаимосвязанных классов: битового поля (TBitField) и множества (TSet), основанного на использовании битовых полей в качестве характеристических векторов.

Битовое поле представляет собой эффективную структуру данных, позволяющую компактно хранить и обрабатывать логические значения в памяти. Битовое поле производит упаковку bit независимых бит (где значение bit автоматически вычисляется через sizeof(TELEM) * 8 и составляет 32 бита для типа unsigned int).

Множество — математический объект, элементами которого в данной работе выступают неотрицательные целые числа из ограниченного диапазона. Соответствие между множеством и битовым полем строится через концепцию характеристического вектора: i-й бит поля равен 1, если число i входит в состав множества, и равен 0 в обратном случае.

1. Постановка задачи

На основе предоставленных заголовочных файлов интерфейсов, находящихся в папке includes, требуется реализовать:

1. Класс **TBitField** со следующим набором обязательных операций:
   - Конструктор по заданной длине поля, конструктор копирования, деструктор
   - Методы получения логической длины (GetLength), установки бита (SetBit), сброса бита (ClrBit) и считывания его текущего значения (GetBit)
   - Перегруженные операторы сравнения (==, !=), присваивания (=), а также базовые побитовые операции: побитовое ИЛИ (|), побитовое И (&), побитовое отрицание (~)
   - Потоковые операторы ввода и вывода объектов (>>, <<).
2. Класс **TSet** со следующим набором обязательных операций:
   - Конструктор по максимальной мощности множества, конструктор копирования, а также конструктор явного преобразования типа на базе готового объекта TBitField
   - Оператор явного приведения типа к TBitField
   - Методы получения мощности (GetMaxPower), включения элемента в состав множества (InsElem), удаления элемента (DelElem), проверки его членства (IsMember);
   - Перегруженные теоретико-множественные операторы: объединение с элементом и вторым множеством (+), разность с одиночным элементом (-), пересечение множеств (*), дополнение множества до универса (~);
   - Операторы логического равенства (==, !=), оператор присваивания (=), а также функции ввода и вывода в стандартные потоки данных.

Также необходимо написание автотестов **Google Test**, полностью покрывающих функционал и все краевые случаи работы классов.

2. Руководство пользователя

Всё взаимодействие пользователя с кодовой базой происходит посредством выполнения автоматизированных тестов Google Test.

Порядок компиляции и запуска тестов в среде Microsoft Visual Studio:

1. Открыть файл решения (.sln) лабораторной работы в среде разработки.
2. Произвести полную сборку решения.
3. Результаты выполнения (успешный проход или логических ошибок на конкретных строках) отобразятся в консоли.

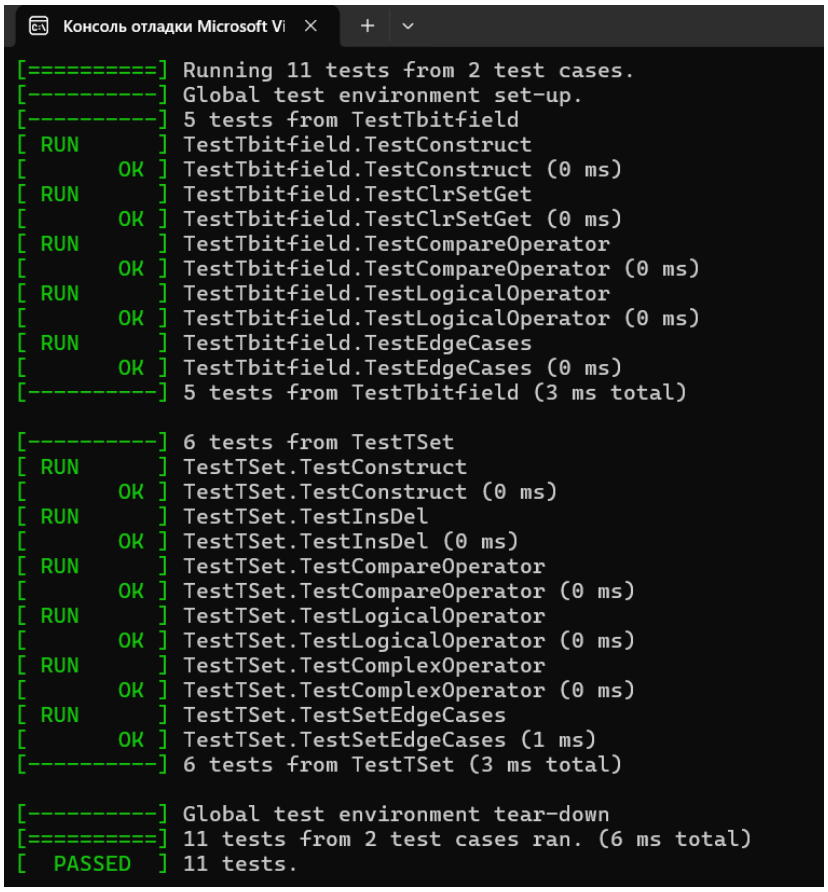


Рис. 2.1. Результат выполнения проекта

3. Руководство программиста

3.1. Описание структуры программы

Проект логически разбит на изолированные модули:

• **tbitfield.h** — внешний заголовочный файл с объявлением интерфейса класса битового поля.

• **TBitField.cpp** — файл реализации внутренней логики и методов класса TBitField.

• **tset.h** — заголовочный файл с объявлением интерфейса класса множества.

• **TSet.cpp** — файл реализации внутренней логики, методов и операторов класса TSet.

• **TestTBitfield.cpp** — исходный код автоматизированных тестов для проверки битовых полей.

• **TestTSet.cpp** — исходный код автоматизированных тестов для проверки множеств.

3.2. Описание структур данных

Класс **TBitField** имеет три приватных поля:

- int BitLen — количество бит в поле
- int MemLen — физический размер динамического массива (количество ячеек типа TELEM - беззнаковых целых чисел типа unsigned int (переименован через typedef unsigned int TELEM))
- TELEM* pMem — указатель на динамически выделенный массив из типа данных TELEM.

Адресация конкретного бита в непрерывном массиве осуществляется при помощи двух приватных вспомогательных методов:

- GetMemIndex(n) — вычисляет индекс ячейки массива pMem, где лежит бит: (n / bit)
- GetMemMask(n) — генерирует локальную битовую маску внутри найденной ячейки: 1 << (n % bit).

Класс **TSet** имеет следующие приватные поля:

- int MaxPower — максимальное количество элементов (универсум множества)
- TBitField BitField — объект битового поля, выступающий в роли характеристического вектора.

3.3. Описание алгоритмов

SetBit(n), ClrBit(n), GetBit(n): Каждый из методов в первую очередь производит валидацию входного индекса: если (n < 0) или (n >= BitLen), генерируется исключение std::out_of_range. Установка бита выполняется через побитовое ИЛИ (pMem[index] |= mask).

Побитовое ИЛИ (operator|), Побитовое И (operator&): Операции создают новое результирующее битовое поле, чья длина равна максимуму из длин двух исходных полей. Цикл обрабатывает общую часть массивов (до минимального значения MemLen), применяя соответствующие побитовые операции (| или &). Оставшийся "хвост" более длинного поля (для операции ИЛИ) копируется в результат без изменений.

Оператор присваивания (**operator=**) включает проверку на самоприсваивание (this != &bf). Если физические размеры массивов (MemLen) не совпадают, старая динамическая память очищается через delete[], после чего выделяется новый блок памяти нужного размера, обновляются параметры длин и данные копируются поэлементно.

Теоретико-множественные операции (**operator+, operator***): Перед выполнением объединения или пересечения битовых полей в обязательном порядке сравниваются их мощности MaxPower. При попытке сопоставить множества из разных универсумов генерируется исключение std::invalid_argument. Если мощности равны, задача переходит к операторам битового поля (| или &).

Побитовое отрицание (**operator~**): Создается результирующее битовое поле той же длины. Алгоритм последовательно инвертирует каждый бит исходного поля в цикле от 0 до BitLen - 1. Если метод GetBit(i) возвращает 0, то в новом поле вызывается метод SetBit(i). Это позволяет получить результат без мусорных бит в неиспользуемом остатке последней ячейки памяти.

Добавление и удаление элемента множества (**InsElem, DelElem**): Методы производят валидацию переданного числа Elem на попадание в допустимый диапазон универсума [0, MaxPower - 1]. При выходе за границы генерируется исключение std::out_of_range. При успешной валидации задача транслируется на уровень характеристического вектора: вызываются методы BitField.SetBit(Elem) или BitField.ClrBit(Elem) соответственно.

Объединение и пересечение множеств (**operator+, operator***):

1. Перед побитовой обработкой проверяется совместимость операндов: мощности MaxPower текущего множества и множества s должны быть строго равны. При несовпадении универсумов генерируется исключение std::invalid_argument.

2. В случае равенства мощностей алгоритм выполняет операцию над битовыми полями (BitField | s.BitField или BitField & s.BitField). Результат передается в

конструктор преобразования типов, возвращающий новое валидное множество TSet.

Потоковый вывод битового поля TBitField: Алгоритм считывает логическую длину поля через bf.GetLength(). Затем в цикле от 0 до len - 1 последовательно опрашивается каждый бит с помощью метода bf.GetBit(i). Если метод возвращает истину, в выходной поток stream записывается символьный литерал '1', в противном случае — '0'. По завершении цикла поток возвращается для обеспечения возможности построения цепочки вызовов.

Потоковый ввод битового поля для TBitField:

1. Алгоритм считывает логическую длину целевого объекта bf.GetLength() и предварительно сбрасывает все его биты в нулевое состояние в цикле с помощью bf.ClrBit(i).

2. Затем в цикле выполняется посимвольное чтение из входного потока stream в переменную elem.

3. Если считан символ '1', вызывается метод bf.SetBit(i), если '0' — бит остается нулевым (или принудительно вызывается bf.ClrBit(i)).

4. При обнаружении любого другого некорректного символа алгоритм выводит сообщение "Input error"

Потоковый вывод множества для TSet: Алгоритм последовательно перебирает все потенциальные элементы универсума в цикле от 0 до s.GetMaxPower() - 1. На каждой итерации членство элемента проверяется вызовом s.IsMember(i). Если элемент входит в множество, его числовое значение выводится в поток stream, сопровождаясь пробельным разделителем для предотвращения слипания чисел. В конце вывода добавляется символ переноса строки '\n'.

Потоковый ввод множества TSet:

1. Перед началом заполнения полностью очищается текущее состояние множества, удаляя все элементы в цикле от 0 до s.GetMaxPower() - 1 с помощью метода s.DelElem(i).

2. Запускается бесконечный цикл чтения целочисленных элементов elem из потока stream.

3. Если чтение прерывается из-за некорректного формата срабатывает проверка stream.fail(), которая выводит сообщение "Input error", сбрасывает флаг ошибки через stream.clear(), игнорирует ошибочную строку в буфере и переходит к следующей итерации.

4. При успешном считывании числа проверяется маркер окончания ввода: если elem < 0, цикл завершается.

5. Для корректных неотрицательных чисел выполняется проверка границ, после чего элемент успешно добавляется в множество вызовом s.InsElem(elem).

4. Результаты экспериментов и тестирования

Для верификации разработанных классов были запущены 11 комплексных тестовых сценариев Google Test, разделенных на два набора: TestTbitfield и TestTSet.

Список тестов набора TestTbitfield:

1. TestConstruct: Проверка корректности работы конструктора по умолчанию, инициализации логической длины поля, а также конструктора копирования и оператора присваивания.
2. TestClrSetGet: Валидация методов побитового изменения данных. Проверяется точечное включение бита, его считывание и гарантированный сброс обратно в 0.
3. TestCompareOperator: Проверка корректности работы логических операторов равенства == и неравенства != для полей одинаковой размерности.
4. TestLogicalOperator: Тестирование побитовых операций объединения (|), пересечения (&) и инверсии (~) с проверкой полей на уровне циклов.
5. TestEdgeCases: Проверка граничных и аварийных сценариев. Проверяется генерация исключений std::out_of_range при создании полей отрицательной длины и при попытке деструктивного доступа к индексам -1, 10, 100 на поле фиксированного размера 10. Проверено поведение оператора присваивания при переходе от больших размеров к меньшим.

Список тестов набора TestTSet:

1. TestConstruct: Проверка создания множества, работы конструктора преобразования типов из TBitField во множество и обратной операции неявного приведения типа.
2. TestInsDel: Валидация вставки элементов в множество (InsElem) и их удаления (DelElem) через контроль битовой маски характеристического вектора.
3. TestCompareOperator: Проверка эквивалентности множеств по составу элементов.
4. TestLogicalOperator: Проверка стандартных операций объединения и пересечения. Проверяется генерация исключения std::invalid_argument при попытке математического взаимодействия множеств разной мощности.
5. TestComplexOperator: Контроль корректности выполнения приоритетных цепочек выражений, например: (a + 5) * b - 2, с проверкой неизменяемости исходных операндов.
6. TestSetEdgeCases: Проверка устойчивости к самоприсваиванию множеств (self = self), а также перехват исключений при попытках добавить или проверить элементы вне установленных границ универсума множества.

Все 11 автотестов завершились со статусом PASSED

Заключение

В результате работы были успешно решены все поставленные задачи:

1. Реализован класс битового поля TBitField, обеспечивающий компактное хранение данных в динамической памяти (с упаковкой 32 бит в одну ячейку unsigned int).
2. Реализован класс множества TSet, основанный на битовом поле в качестве характеристического вектора. Написаны и отлажены все предусмотренные теоретико-множественные операции.
3. Разработан набор автотестов Google Test, полностью покрывающий функционал и краевые сценарии. Успешное прохождение всех 11 тестовых сценариев со статусом PASSED доказало точность реализованных алгоритмов

Литература

Список источников, используемых при выполнении работы (если есть).

1. Страуструп Бьерн Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.
2. Сайт cppreference: https://en.cppreference.com/

Приложение